part03_cli — 개발 환경과 adk CLI

part03_cli 폴더(= 에이전트들의 상위 폴더)에서 실행한다. 강의노트 **Part 3 · 03. 개발 환경과 CLI**(슬라이드 30–48)와 짝을 이룬다.

```
part03_cli/
├── my_agent/              # 📁 ADK 기본 에이전트 골격 (프롬프트 질의응답)
├── my_agent_mod/          # 📁 ADK 커스텀 에이전트 (날씨 조회 get_weather 도구 추가)
├── my-agents-cli/         # 📁 agents-cli 기본 엔터프라이즈 템플릿 (FastAPI, SSE, A2A,
├── my-agents-cli-mod/     # 📁 agents-cli 커스텀 에이전트 (실시간 Open-Meteo 날씨 & 세
├── run_direct_runner.py   # 🔑 직접 Runner 코드: 등록 → 실행 → 이벤트 소비 (슬라이드 23–
├── adk_internals.py       # 🟢/🔑 adk 가 대신 해 주는 5단계 재현 (슬라이드 31–38)
├── adk_cli_map.py         # 🟢 adk CLI 소스 구조 확인: 명령 트리 · AgentLoader · URI s
├── api_client.py          # 🔑 adk api_server 의 /run · /run_sse 호출 (슬라이드 43)
├── _path.py               # ⚙️ 상위 디렉터리 및 패키지 경로 탐색 헬퍼
├── README-KR.md           # 📄 전체 실습 한글 가이드 (본 파일)
└── README-EN.md           # 📄 전체 실습 영문 가이드
```

.env는 실습 루트(ADK_Labs/.env) 하나만 두면 된다. adk가 에이전트 폴더에서 상위로 올라가며 찾는다.

📂 서브 폴더 개요 및 요약

서브 폴더	도구/프레임워크	핵심 내용	주요 실행 방법
my_agent	adk CLI	adk create my_agent로 생성되는 가장 순수한 ADK 기본 에이전트. 도구 없이 Gemini 본연의 언어 모델 추론을 통해 질문에 답변.	adk run my_agent adk web --port 8000
my_agent_mod	adk CLI	기본 ADK 구조에 로컬 함수 get_weather 도구를 추가(Function Calling)한 도시 날씨 조회 에이전트.	adk run my_agent_mod adk web --port 8000
my-agents-cli	google-agents-cli	agents-cli create로 생성된 프로덕션급 템플릿. FastAPI 기반 백엔드, SSE 스트리밍, A2A 프로토콜, Reasoning Engine 연동, 테스트(단위/통합/평가), Terraform 배포 코드 포함.	cd my-agents-cli agents-cli playground agents-cli install
my-agents-cli-mod	google-agents-	my-agents-cli의 app/agent.py를 고도화하여 실제 외부 Open-Meteo REST API(날씨/지오코딩)	cd my-agents-cli-mod

서브 폴더	도구/프레임워크	핵심 내용	주요 실행 방법
	cli	및 IANA 표준 타임존 기반 실시간 세계 시각 조회를 지원하는 완성형 에이전트.	agents-cli playground

1) my_agent/ 요약

- **구조**: agent.py, __init__.py, .env, .gitignore, .adk/session.db
- **역할**: 최소 단위의 ADK 기본 에이전트(gemini-3.5-flash). 별도 도구 없이 LLM의 기본 시스템 지침을 수행.
- **실행**: part03_cli 폴더에서 adk run my_agent 또는 adk web --port 8000

2) my_agent_mod/ 요약

- **구조**: agent.py, __init__.py, .env, .gitignore, .adk/session.db
- **역할**: get_weather(city: str) 로컬 도구가 등록된 날씨 특화 에이전트(weather_agent).
- **실행**: part03_cli 폴더에서 adk run my_agent_mod 또는 adk web --port 8000

3) my-agents-cli/ 요약

- **구조**: app/ (FastAPI, agent, A2A, Reasoning Engine), tests/ (unit, integration, eval), deployment/terraform/, pyproject.toml, Dockerfile 등
- **역할**: Mock 도구(get_weather, get_current_time)와 엔터프라이즈 기능(A2A 프로토콜, 평가 파이프라인, Terraform 인프라 배포)이 완비된 기본 템플릿.
- **실행**: my-agents-cli/ 내부에서 agents-cli playground 또는 uv run uvicorn app.fast_api_app:app --host 0.0.0.0 --port 8000

4) my-agents-cli-mod/ 요약

- **구조**: my-agents-cli 구조 + app/agent.py 실전 API 연동 + 웹 아키텍처 가이드 PPT
- **역할**: Open-Meteo API 실시간 날씨 및 전 세계 도시 시간 조회 기능 탑재, 오프라인 모드(AGENT_OFFLINE=1) 폴백 제공.
- **실행**: my-agents-cli-mod/ 내부에서 agents-cli playground

---

1. 같은 에이전트, 두 가지 실행 방식 (슬라이드 23–25 · 31–39)

ADK 응용 SW의 뼈대는 Runner다. 같은 my_agent/agent.py를 두 방식으로 실행해 비교한다.

방식	개발자가 쓰는 것	실습
직접 Runner 코드	root_agent + Runner · 서비스 · 입출력 코드	run_direct_runner.py

방식	개발자가 쓰는 것	실습
adk CLI	root_agent(agent.py)만	adk run my_agent · adk_internals.py

① 직접 Runner 코드: 등록 → 실행 → 이벤트 소비 (🔑)

```
uv run run_direct_runner.py "부산 날씨 알려줘"
```

```
① 등록   Runner(agent=weather_agent, session_service=InMemorySessionService)
② 실행   run_async(new_message='부산 날씨 알려줘')
   [Event 1] 모델 호출 → function_call get_weather({'city': '부산'})   (모델은 요청만)
   [Event 2] 도구 실행 → function_response get_weather: {'status': 'success', 'repor
③ 소비   최종 응답(Event 3): 부산의 현재 날씨는 흐리고 기온은 20°C입니다.
```

`run_async()` 한 번이 agent loop 한 바퀴다(슬라이드 25): 모델이 function_call을 "요청"하면 Runner가 도구를 실행하고 결과로 모델을 다시 부른다.

② adk 가 대신 해 주는 일 (🟢 키 없이 ①~④, 🔑 ⑤)

`adk run my_agent`가 내부에서 하는 5단계를 ADK 자신의 코드(`AgentLoader`, `load_dotenv_for_agent`, `create_*_service_from_options`)로 그대로 재현한다.

```
python .\adk_internals.py                                          # 기본: ①~④ 재현
python .\adk_internals.py --in_memory                              # 메모리 서비스 사용
python .\adk_internals.py --session_service_uri sqlite:///tmp_sessions.db
python .\adk_internals.py --ask "서울 날씨?"                        # 실제 LLM 호출로 한 턴
```

기본 실행은 `my_agent`를 찾아 `.env`를 로드하고, 세션·아티팩트·메모리 서비스를 만든 다음 `Runner`를 구성해 각 단계의 정보를 출력합니다. `--ask`를 주지 않으면 여기서 종료하며, 사용자 입력을 기다리는 REPL이나 서버를 시작하지 않고 LLM도 호출하지 않습니다. 따라서 기본 실행은 LLM API 키 없이 동작합니다.

`--in_memory`는 세션과 아티팩트에 메모리 기반 서비스를 선택합니다. `--session_service_uri`는 세션 저장소를 직접 지정합니다. 두 옵션은 이 스크립트가 구성하는 서비스에 영향을 줍니다.

`--ask "질문"`을 지정하면 새 세션을 만들고 해당 질문을 `runner.run_async()`에 전달해 한 턴을 실행합니다. 모델 응답과 도구 호출 정보를 출력하며, 실제 LLM 호출이므로 API 키와 모델 설정이 필요합니다.

```
① 찾기          AgentLoader → LlmAgent name='weather_agent' (from my_agent/agent.py)
② .env 로드     GOOGLE_API_KEY 설정됨=True · ADK_MODEL=gemini-3.5-flash-lite
③ 서비스 생성   session=PerAgentDatabaseSessionService → my_agent/.adk/session.db (SQL
                artifact=PerAgentFileArtifactService · memory=InMemoryMemoryService
④ Runner 생성   Runner(app='my_agent', root_agent='weather_agent')
⑤ 입구 제공     adk run 이면 여기서 input('[user]: ') 루프 → runner.run_async()
```

- **기본 세션 저장소는 InMemory가 아니다.** 플래그가 없으면 에이전트 폴더의 `.adk/session.db`(SQLite)에 저장된다. `--in_memory`를 주거나 Cloud Run·Kubernetes에서 실행되면 InMemory로 바뀐다. 운영에서는 `--session_service_uri`로 지정한다.
- `adk web`·`adk api_server`도 같은 서비스·Runner를 만들고, 차이는 ⑤ 입구(HTTP)뿐이다(슬라이드 38).
- `google.adk.cli.utils`는 ADK 내부 API라 버전마다 바뀔 수 있다(2.6.3 기준 학습용 재현).
- `adk run`·`adk_internals.py`를 실행하면 `my_agent/.adk/` 폴더가 생긴다. 지워도 된다.

---

2. agents-cli (슬라이드 44–48, 상세는 Part 13 슬라이드 143–156)

agents-cli 소개 · adk와의 비교 · 명령 지도 · 프로젝트 구조 · 동작 구조 실습은 part13_agents_cli/에 있다.

---

3. adk CLI 동작 구조 (슬라이드 32–36)

`google/adk/cli` 소스(google-adk 2.14.0 = GitHub main)를 슬라이드와 맞춰 직접 확인한다. 키 없이 실행된다.

```
python adk_cli_map.py            # commands · loader · services · routes
```

항목	슬라이드	확인할 것
commands	32	adk = cli_tools_click.main Click 그룹, 명령마다 구현 함수
loader	34	AgentLoader.load_agent('my_agent') → LlmAgent
services	34	세션 memory · sqlite · postgresql · mysql · agentengine, 아티팩트 memory · gs · file …
routes	35–36	api_server 경로 22개 · adk web 59개(/dev/* 35개)

adk_cli_map.py 실행 시 수행되는 일

```
python .\adk_cli_map.py                 # 네 항목 전체 확인
python .\adk_cli_map.py commands        # 명령 트리만 확인
python .\adk_cli_map.py loader          # 에이전트 로더만 확인
python .\adk_cli_map.py services        # 서비스 URI scheme만 확인
python .\adk_cli_map.py routes          # API 경로만 확인
```

인자를 생략하면 `all`과 동일하게 아래 네 항목을 차례로 출력합니다.

- **commands**: 설치된 ADK의 Click 명령 그룹을 순회해 adk 명령 트리와 각 명령의 처리 함수를 출력합니다.
- **loader**: `AgentLoader`가 `part03_cli`에서 `my_agent`를 찾아 로드한 객체의 종류와 이름, 에이전트 검색 순서를 출력합니다. 에이전트 정의를 로드하는 것이며 사용자 질의를 실행하지는 않습니다.
- **services**: ADK 서비스 레지스트리에서 session, artifact, memory 서비스가 지원하는 URI scheme을 출력합니다.
- **routes**: `api_server`용(`web=False`)과 `adk web`용(`web=True`) FastAPI 앱을 메모리 안에서 구성하고, 각 앱의 경로 수, `/run*` 경로, `/dev/*` 경로 수 및 web 전용 경로 예시를 출력합니다. 이 과정에서 서버를 시작하거나 에이전트 질의를 보내지 않습니다.

이 스크립트는 ADK 내부 구현을 학습하기 위한 조회용 도구입니다. LLM 질의를 하지 않으므로 API 키가 필요하지 않으며, 출력 경로 수와 명령 목록은 설치된 ADK 버전에 따라 달라질 수 있습니다.

---

4. adk 주요 명령 (슬라이드 40–43)

명령	하는 일
`adk create my_agent2`	agent.py · __init__.py · .env 골격 생성
`adk run my_agent`	터미널 REPL (예: 서울 날씨 알려줘)
`adk run --save_session --session_id s1 my_agent`	종료 시 `my_agent/s1.session.json` 저장 → eval 데이터로 재활용
`adk run --resume my_agent/s1.session.json my_agent`	저장한 대화를 이어서 진행
`adk run --replay queries.json my_agent`	`{"state": {...}, "queries": [...]}` 입력 파일을 사람 입력 없이 실행 (세션 파일이 아니다)
`adk web --port 8000`	개발 UI: http://localhost:8000 (Events · Trace · State · Graph 탭)
`adk api_server --port 8000 --session_service_uri="sqlite:///sessions.db"`	헤드리스 REST/SSE 서버

플래그는 버전마다 다를 수 있으니 `adk run --help`로 확인한다.

5. api_server 호출

part03_cli 폴더에서 adk api_server를 실행하면 하위 폴더의 에이전트들을 API로 제공합니다. 서버를 시작하는 것만으로 특정 에이전트가 실행되지는 않습니다. /run 또는 /run_sse 요청의 app_name 값이 호출할 에이전트를 선택합니다. 사용 가능한 에이전트는 GET http://localhost:8000/list-apps로 확인할 수 있습니다.

```
adk api_server --port 8000
```

다른 터미널에서:

```
uv run api_client.py
```

api_client.py의 기본값은 APP = "my_agent"입니다. 날씨 도구가 포함된 에이전트를 호출하려면 APP = "my_agent_mod"로 변경하세요. curl 요청도 같은 방식으로 app_name과 세션 경로의 에이전트 이름을 지정합니다.

/run과 /run_sse의 차이

두 엔드포인트 모두 같은 에이전트에 질의를 보내지만 이벤트를 전달하는 방식이 다릅니다.

엔드포인트	응답 방식	클라이언트 사용
POST /run	에이전트 실행이 끝난 뒤 전체 이벤트를 JSON 배열로 한 번에 반환	완료 후 배열을 순회해 이벤트를 처리
POST /run_sse	실행 중 생성되는 이벤트를 Server-Sent Events(SSE) 스트림으로 전달	연결을 유지하며 이벤트를 순차적으로 읽고 처리

따라서 /run은 전체 결과를 한꺼번에 받을 때, /run_sse는 실행 도중 이벤트를 실시간으로 소비하거나 진행 상황을 보여줄 때 적합합니다. api_client.py는 먼저 /run으로 서울 날씨? 요청의 전체 이벤트를 받은 다음, 같은 세션에서 부산은? 요청을 /run_sse로 보내 이벤트를 한 줄씩 출력합니다. 두 요청은 각각 별도의 에이전트 턴입니다.

curl로 직접 호출할 때:

```
# 세션 생성
curl -X POST localhost:8000/apps/my_agent_mod/users/u1/sessions/s1

# 에이전트 질의 실행
curl -X POST localhost:8000/run -H "Content-Type: application/json" -d "{\"app_name
```